

Plan Maestro de Orquestación Multi-Agente

Proyecto: Hotel Wimbledon (Sistema Integral de Gestión & Check-in QR)

Arquitectura del Proyecto:

- **Backend:** Java 21 / Spring Boot 3.5 (Maven)
- **Frontend:** React / Vite / Vanilla Admin
- **Persistencia:** MySQL 8.x + JPA / Hibernate
- **Módulos:** Rack Habitaciones, QR, RBAC (5 roles)

Infraestructura de Agentes:

- **Orquestador:** Gentle-AI 3.7.0 (Framework SDD)
- **Constructor:** OpenCode 2.0.16 (Space Bunny Free)
- **Auditor/Adversario:** Hermes Agent 0.21.5
- **Memoria Persistente:** Engram 2.2.0 (MCP Server)

Fecha de Emisión: 25 de Septiembre de 2026

Estado: Validado y Operativo en Entorno Local (Arch / CachyOS Linux)

Presupuesto de Inferencia: \$0.00 USD (Modelos Gratuitos Remotos + LM Studio Local)

1 Resumen Ejecutivo y Objetivos

El presente documento establece el marco operativo y técnico para coordinar de forma sinérgica el conjunto de agentes de inteligencia artificial aplicados al repositorio **Hotel Wimbledon** (`wimbledon-web`).

El objetivo primordial es reemplazar la programación reactiva o monolítica por un ciclo de vida estructurado bajo la metodología **SDD (Spec-Driven Development)**, implementada a través del framework **Gentle-AI**. En este esquema:

- Especialización Estricta:** Ningún agente asume roles cruzados sin supervisión. El constructor implementa código, el auditor desafía la seguridad y la usabilidad, y el orquestador valida cada fase antes de avanzar.
- Cero Costo en Inferencia (Zero API Cost):** Se integran modelos en la nube de nivel de producción sin consumo financiero (`Space Bunny Free` y `Nous Laguna 2.1 Free`), respaldados por el servidor local **LM Studio** para contingencias fuera de línea.
- Memoria Trans-Sesión (Cross-Session Resilience):** Se garantiza que decisiones clave de arquitectura, esquemas de base de datos y hallazgos adversariales persistan frente a la compactación de contexto mediante el protocolo MCP de **Engram**.

2 Topología de la Arquitectura de Agentes

El flujo de trabajo divide las responsabilidades en cuatro capas jerárquicas:



3 Matriz de Roles y Reglas de Compromiso

El equipo opera bajo el principio de desconfianza mutua y límites claros, tal como se especifica en `.agents/AGENTS.md` del proyecto Wimbledon:

Agente	Entorno	Enfoque Principal	Archivos / Objetivos Clave	Restricción Crítica
Gentle Orchestrator	OpenCode CLI / TUI	Descomposición SDD y verificación de hitos	.specs/, tareas SDD, contratos de revisión	No codifica directamente sin fase previa aprobada.
OpenCode (Builder)	OpenCode 2.0.16	Implementación de código productivo	backend/src/... frontend/src/...	No realiza auto-aprobación de seguridad ni merge.
RedTeam	Hermes Agent / Sandbox	Pentesting, bypass de RBAC, inyección JWT	SecurityConfig.java JwtAuthFilter.java @PreAuthorize Matriz de 5 Roles	No modifica código fuente; solo emite reportes con severidad.
UI-Critic	Hermes Agent / Visual Subagent	Consistencia de Rack, a11y, fricción de reserva	index.html (Público) admin.html (Panel) style.css (Variables) BPMN To-Be	No evalúa backend ni infraestructura; foco 100% experiencia.
Hermes Runner	Hermes CLI (-w)	Ejecución de pruebas, validación CLI y logs	mvn test, curl endpoints, verificación de estado MySQL	Opera en git worktrees aislados para evitar conflictos.

Regla de Oro de Interacción:

Ningún cambio que afecte endpoints de recepción, cobro o autenticación (/api/auth, /api/reservas, /api/admin) podrá considerarse completo hasta que el reporte del agente RedTeam certifique ausencia de brechas de nivel **Crítico** o **Alto**, y el agente UI-Critic apruebe la coherencia contra el diagrama BPMN To-Be.

4 Fases de Aplicación en el Proyecto Hotel Wimbledon

A continuación se detalla la secuencia operativa para abordar cualquier requerimiento o refactorización en el sistema de Wimbledon:

4.1 Fase 1: Inicialización y Especificación (SDD Init & Spec)

- **Comando:** `/sdd-init <nombre-cambio>` en OpenCode o `gentle-ai sdd-status`.
- **Acción:** El orquestador crea el directorio de trabajo bajo especificación controlada.
- **Rol del Humano:** Definir los requisitos funcionales del cambio (por ejemplo: «*Implementación de firma digital en check-in con QR*»).
- **Salida:** `proposal.md` y `spec.md` con alcance delimitado y criterios de aceptación verificables.

4.2 Fase 2: Diseño de Tareas y Arquitectura (SDD Design & Tasks)

- **Acción:** `gentle-orchestrator` descompone la especificación en unidades de trabajo atómicas (`tasks.md`).
- **Persistencia:** Se invoca `mem_save` en Engram con los identificadores de endpoints y modelos de datos afectados (ejemplo: `HabitacionDTO`, `CheckInRequest`).
- **Revisión:** Se valida que no existan contradicciones con las reglas de negocio de los 5 roles existentes (`SUPER_ADMIN`, `ADMINISTRADOR`, `RECEPCIONISTA`, `LIMPIEZA`, `CLIENTE`).

4.3 Fase 3: Construcción Aislada (OpenCode Apply)

- **Herramienta:** OpenCode sobre el modelo `space-bunny-free`.
- **Tareas:**
 - Generación de código backend en Spring Boot (`*Service.java`, `*Controller.java`).
 - Creación de migraciones SQL para MySQL 8 si se añaden campos.
 - Implementación de la vista interactiva en Vite / React o Vanilla JS (`admin.js`).
- **Principio:** Compilación e integración continua local (`mvn clean compile`).

4.4 Fase 4: Auditoría Cruzada Dual (Hermes + RedTeam + UI-Critic)

- **Herramienta:** Hermes Agent ejecutando de forma independiente sobre `poolside/laguna-s-2.1:free`.
- **Ejecución de RedTeam:**
 1. Envía peticiones HTTP anómalas contra los endpoints recién expuestos.
 2. Intenta acceder con tokens de rol `CLIENTE` o `LIMPIEZA` a endpoints reservados para `RECEPCIONISTA` o `ADMINISTRADOR`.
 3. Verifica que las excepciones retornen respuestas HTTP 401/403/400 estandarizadas y sin volcado de stack trace.
- **Ejecución de UI-Critic:**
 1. Comprueba la paleta de colores y tokens (`--color-gold`, `--color-navy`, tipografía Fraunces).
 2. Valida la usabilidad del Rack de Habitaciones ante cambios de estado (`LIBRE`, `OCUPADA`, `LIMPIEZA`).
 3. Comprueba el flujo de lectura de QR en dispositivos móviles y de escritorio.

4.5 Fase 5: Verificación, Cierre y Registro en Memoria

- **Comando:** `/sdd-verify` y `/sdd-archive`.
- **Acción:** El orquestador ejecuta los tests unitarios y de integración (`mvn test`).
- **Cierre en Engram:** Se almacena el resumen final de la sesión mediante `mem_session_summary`, vinculando el cambio a la historia del proyecto Wimbledon.

5 Protocolo de Manejo de Inferencia y Fallbacks

Escenario	Proveedor Primario	Fallback Inmediato	Condición de Activación
OpenCode (Desarrollo diario)	opencode/space-bunny-free (OpenRouter / OpenCode de Pool)	LM Studio Local (prism-ml/bonsai-27b o qwen3.5-9b)	Agotamiento de cuota, latencia > 15s o pérdida de conexión externa.
Hermes Agent (Auditoría / Tareas)	nous/poolside/laguna-s-2.1:free (Nous Portal Inference)	LM Studio Local (google/gemma-4-12b-qat)	Errores HTTP 429/503 en Nous Portal.
Inferencia Local (LM Studio)	Aceleración CUDA (RTX 4070 Ti Super)	Offload CPU (0 capas GPU)	Cuando procesos externos (ej. Python PID 190164) ocupen la VRAM.

6 Hoja de Ruta de Casos de Uso en Wimbledon

1. **Hito 1 — Fortalecimiento del Check-in QR:**
 - **Objetivo:** Permitir validación criptográfica del código QR emitido en la reserva para check-in express sin intervención manual de recepción.
 - **Asignación:** OpenCode (Backend ZXing + Endpoint /api/recepcion/checkin-qr), RedTeam (Auditoría de replay attacks con QR ya canjeados).
2. **Hito 2 — Rack Interactivo de Habitaciones en Tiempo Real:**
 - **Objetivo:** Refrescar el estado de ocupación y limpieza en admin.html mediante eventos SSE o polling optimizado.
 - **Asignación:** OpenCode (Frontend DOM updates), UI-Critic (Validación ergonómica y prevención de parpadeos en pantalla).
3. **Hito 3 — Blindaje de Auditoría de Transacciones de Caja:**
 - **Objetivo:** Registro inmutable de pagos en recepción (EFECTIVO, TARJETA, YAPE/PLIN).
 - **Asignación:** OpenCode (Entidad Pago + Servicio), RedTeam (Pruebas de concurrencia y doble cobro).